

Project 2 面试导向完善与演化路线

当前修改主线 + 后续 Agent / Serving 技术演化

项目定位：银行业务 RAG / Controlled Agentic RAG，用于求职展示与系统化学习，不以生产级完备为目标

文档版本：v1.2（由 v1.1 长期路线压缩为“当前必须做 + 后续演化”两部分）

更新日期：2026-10-01

1. 这版路线的核心变化

旧版路线偏向“长期可维护的企业级系统”；新版把项目目标改为“能在面试中讲清技术演化、实验依据和关键工程能力”。

保留的原则

冻结自然 Baseline；复杂问题单独验证；先证明失败模式，再引入技术；每个新技术都要有进入理由和可验证收益。

删减的原则

不为了代码整洁做大重构；不追求完整生产级数据治理、全量可观测性或高并发服务；招聘 JD 中的热门技术可以进入项目，但只在能形成合理技术故事时进入。

PART I

当前必须完成的修改

2. 当前事实基线：先承认 Baseline 已经很好

模块	已验证结果	当前判断
Hybrid Retriever	Hit@5 94%；Complete@5 92%； Complete@20 100%	主检索链路稳定，保留
Cross-Encoder Reranker	Hit@5 100%；Complete@5 96%； MRR@5 0.824	能拉回后排证据，但多证据完整性仍可能被破坏

模块	已验证结果	当前判断
Minimal Pipeline	Complete@5 96%; MRR@5 0.824	当前默认路径
Low Planner Pipeline	Complete@5 96%; MRR@5 0.799	没有证明净收益
最新一次延迟	Minimal 1.657s; Low 11.122s	延迟受环境影响，但 Low 明显更慢

现在真正的问题

不是“怎么把 96% 再硬刷高”，而是：剩余失败是否集中在真正需要多证据覆盖的复杂 Query？如果是，Query Decomposition 能否只在这类问题上带来净收益？

3. 当前第一优先级：新增 Planner 专项评测，而不是修改旧测试集

原 61 条 bank_eval_natural_v1 永久冻结，用于回归测试；另建复杂问题集来验证 Planner。首版不追求“统计学完美”，只要足够暴露 Planner 的能力差异。

数据集	建议规模	用途
natural_v1	现有 61 条，不改	检查旧能力是否退化；所有新版本都跑
planner_dev	建议首版 20-30 条	跨段、跨协议、多条件、多信息需求；允许调 Prompt / Fusion
planner_test_locked	建议首版 10-15 条	锁定验收；不参与调参

复杂问题应覆盖的典型形式

- 多个独立信息需求：例如“银行卡丢了怎么办？挂失后还能不能存钱？”
- 跨段 / 跨协议证据：答案必须同时引用不同 Chunk 才完整。
- 条件组合：不同身份、时间、产品状态会改变答案。
- 比较型问题：需要分别检索 A / B 后再比较。
- 明确要求多证据完整性：少一条关键证据就算失败，而不是 Hit 到任意一条就算成功。

每条样本最小字段

- query
- task_type / evidence_mode
- required_evidence（必需证据集合）
- expected_points（答案必须覆盖的要点）
- gold_subqueries（复杂问题建议人工给一版 Oracle 拆分）

4. 当前第二优先级：用 Oracle 实验定位 Planner 到底哪里有问题

不要继续盲调 Planner Prompt。先用同一套 Retriever、Reranker、candidate_k 和最终 K，做严格对照。



方案	做法	回答的问题
Original Query	现有 Minimal Baseline	复杂问题本来能做到什么程度
LLM Planner	当前 Query Planner 拆分	真实 Planner 的收益 / 负优化
Oracle Subqueries	人工给出理想拆分	判断 Query Decomposition 这条路本身有没有上限收益

必须保留的链路日志

- 每个 Subquery 的 Hybrid Top20
- 每个 Subquery Rerank 后的排名
- Fusion 前证据是否已经找到
- Fusion 后 Top5 是否把必需证据挤掉
- 最终 Complete@5 / MRR / 延迟变化

实验决策树

观察结果	下一步
Oracle 也不优于 Original	停止把 Planner 当主线；说明分解并不是这类问题的主要瓶颈
Oracle 有效，但 LLM Planner 无效	Planner 拆分质量有问题；再考虑 Prompt / Few-shot / LoRA
子查询找到了证据，但 Fusion 后丢失	先改 Fusion / Coverage，不继续调 Planner
复杂问题提升，简单问题退化	Planner 只作为条件路径，不全局替换 Minimal

5. Planner 被证明有效后：引入 Function Calling 做自动路由

为什么 Function Calling 在这里是“有理由的”

用户不会告诉系统“这是简单问题还是复杂问题”。系统需要根据 Query 自动选择一次检索还是 Query Decomposition；这正好可以由 LLM 根据 Tool Schema 的 description 进行语义路由。

建议只暴露两个高层 Tool

Tool	Description 语义	内部逻辑
search_bank_documents	单一信息需求；Original Query → Hybrid → Reranker → TopK	稳定的 Minimal RAG

Tool	Description 语义	内部逻辑
search_bank_documents_with_decom position	多个独立信息需求 / 多证据覆盖; Planner → Multi-query Retrieval → Fusion	Planned RAG

不要把 BM25、Dense、Reranker 拆成三个 Tool 让 LLM 自由编排。底层检索顺序仍由 Python Pipeline 控制，LLM 只决定“走哪条高层能力路径”。

需要评估的 Agent 指标

- Tool 选择准确率 (Simple / Complex routing accuracy)
- Complex Query Recall：该走 Planned 的问题有多少被正确送进去
- Simple Query 误路由率：简单问题有多少被白白送去 Planned
- 整体 Complete@5 / MRR / 延迟 / 调用成本是否有净收益

6. 当前第三优先级：把最终回答闭环做完整，但只做求职需要的深度

能力	最小实现	面试价值
Generation	LLM 只能依据检索证据回答；输出 Expected Points	回答质量可解释
Citation	答案中的关键结论带 Chunk / source 引 用	能证明 Grounded Answer
Refusal	证据不足、超出语料范围时拒答	体现可靠性意识
轻量 Trace	记录 Query、route、subqueries、top chunks、latency	面试能展示 Bad Case 定位

当前不需要做到的东西

完整 OpenTelemetry、生产级审计平台、全量安全攻防、企业级 SLA 都不是当前阻塞项。先保证一条 Query 能被正确路由、检索、生成、引用和拒答。

7. “求职可用 v1” 验收线

- 自然集 Baseline 仍能复现，新增功能不破坏 Minimal。
- 复杂问题集上能明确回答：Planner 有无净收益，瓶颈在 Planner 还是 Fusion。
- 若 Planner 有价值：Function Calling 能自动在 Minimal / Planned 两个 Tool 之间路由。
- 最终答案支持引用；证据不足能拒答。
- 保留 3-5 个有代表性的 Bad Case，能解释修复前后发生了什么。
- 能用 2-3 分钟讲清“为什么加每个模块”，而不是只报技术栈。

PART II

后续学到 Agent / Infra 技术后的项目演化

8. 演化原则：技术点可以进，但必须有“进场理由”

招聘 JD 里的 LangChain、LangGraph、MCP、LoRA/QLoRA、vLLM 都可以融入本项目。判断标准不是“热门不热门”，而是它是否解决一个已经出现的问题，并且能通过实验或工程结果证明价值。

统一进场模板

问题是什么 → 为什么现有方案不够 → 新技术最小实现 → 和旧方案对比 → 收益 / 代价 → 是否保留。

9. LangChain：轻量接入，不重写业务逻辑

项	建议
进入条件	现有 Prompt、Tool Schema、Retriever / Generator 接口开始增多，需要统一组件协议
最小实现	用 LangChain 包装 Prompt、Tool、Structured Output 或 Runnable；底层 Retriever/Reranker 仍复用现有代码
不要做	不要为了“用了 LangChain”重写整个 Pipeline
面试讲法	我先手写核心闭环，确认原理和可测试性，再用框架降低编排代码成本

10. LangGraph：当 Workflow 真正出现状态分支和补检索再迁移

项	建议
进入条件	Function Calling 路由已存在；出现 Minimal / Planned / Refuse、多路状态、Evidence Sufficiency、最多一次补检索
最小实现	把现有 Python 节点包装成 Graph：route → retrieve / plan → fusion → sufficiency → retry_once → generate
需要证明	纯 Python Workflow 与 LangGraph 行为一致；LangGraph 带来更清晰状态、重试或恢复价值
不要做	无限 ReAct Loop、为了画图而画图

11. MCP：当稳定 Tool 需要被外部 Agent / 客户端复用时进入

项	建议
进入条件	search_bank_documents 等 Tool 已稳定，并且希望被 ChatGPT / IDE / 其他 Agent 客户端标准化调用
最小实现	复用已有 Python Function + Tool Schema，增加 MCP Adapter；暴露 search tool 和 document metadata resource
需要证明	普通 Python 调用、Function Calling、MCP 三条入口共用同一核心检索逻辑
面试讲法	Function Calling 解决应用内部工具循环；MCP 解决工具发现和调用协议标准化

12. LoRA / QLoRA：只优化已经被证明的模型能力瓶颈

项	建议
进入条件	Oracle 明显优于 Original，证明 Query Decomposition 有价值；但 LLM Planner 与 Oracle 仍有稳定差距
训练数据	Query → Gold Subqueries；可从 planner_dev + 人工 Oracle 累积
对照	Base + Prompt / Few-shot / LoRA / QLoRA
评估	复杂问题 Complete、格式合法率、简单问题退化、延迟、显存、Locked Test 泛化
止损	Locked Test 没有稳定收益就不保留微调版本

13. vLLM：把它定位成 Serving / 推理优化实验，而不是检索质量优化

项	建议
进入条件	Generator 已稳定，开始关心并发请求、吞吐或 GPU 利用率
最小实现	保持统一 Generator backend；对比 Hugging Face backend 与 vLLM backend
指标	TTFT、TPOT、QPS、显存、并发吞吐
面试讲法	检索质量和推理吞吐是两类问题；vLLM 解决的是 serving efficiency，不是 Retrieval Recall

14. 项目演化历史：面试时按“问题 → 技术”讲，而不是按框架列表讲

阶段	技术形态	主要内容	进入理由
V0 已完成	基础 RAG	PDF → Chunk → Dense Retrieval → LLM	先跑通完整 RAG
V1 已完成	Hybrid Retrieval	BM25 + Dense + RRF	Dense 对关键词 / 条款匹配不稳定
V2 已完成	Cross-Encoder Reranker	候选召回后精排	相关证据召回了但排序不够好
V3 已尝试	Query Planner	复杂 Query 拆 Subqueries	多证据问题 Top5 覆盖不足；首次实验未证明收益
V4 当前	专项评测 + Oracle/Fusion	复杂集、Oracle、链路归因	先证明 Planner 什么时候值得用
V5 条件进入	Function Calling Routing	LLM 在 Minimal / Planned Tool 中选路	用户不需要自己判断 Query 类型
V6 后续	LangGraph	状态化路由、补检索、拒答	Workflow 分支和状态真正复杂后再引入
V7 后续	MCP	标准化暴露稳定 Tool	外部 Agent / 客户端复用能力
V8 条件进入	LoRA / QLoRA	微调 Planner	只有 Planner 模型能力被证明是瓶颈
V9 条件进入	vLLM	Generator Serving benchmark	出现并发 / 吞吐需求后优化推理

15. 明确不优先的内容

- 生产级向量数据库迁移：当前语料规模和写入场景没有必要。
- 完整 corpus / index migration 平台：保留最小版本号 and Hash 即可。
- OpenTelemetry / Phoenix 全链路平台：JSON Trace 足够时不升级。
- Multi-Agent / A2A：当前没有真实多角色、多系统协作需求。
- DPO / PPO / GRPO：没有偏好数据、奖励信号和明确生成行为问题时不进入主项目。
- 为了“看起来工程化”而做大规模目录重构、全量单测或复杂 DevOps。

16. 最终求职故事线（推荐 2 分钟版本）

一句话定位

我从一个基础银行业务 RAG 开始，通过评测和 Bad Case 逐步加入 Hybrid Retrieval、Reranker 和 Query Decomposition；当 Planner 没有带来净收益时，我没有强行保留，而是用复杂问题集和 Oracle 实验定位瓶颈，再让 Function Calling 只在需要时路由到 Planned RAG。后续 LangGraph、MCP、LoRA、vLLM 都只在出现对应状态管理、工具复用、模型能力或推理吞吐瓶颈时进入。

核心标准：技术不是越多越好，而是每个技术都能回答“为什么现在需要它”。